Submitted By:

Mariha Zafar– 2025-CYS-71

Submitted To:

Sir Mujtaba Kamal

For the fulfillment of, Object-Oriented Programming Lab
Department of Computer Engineering University of Engineering and Technology, Lahore

CODE:

```
import java.awt.*;
import java.awt.event.*;
import java.util.HashSet;
import java.util.Random;
import javax.swing.*;

public class PacMan extends JPanel implements ActionListener, KeyListener {

    class Block {
        int x, y, width, height;
        Image image;

        int startX, startY;
        char direction = 'U';
        int velocityX = 0, velocityY = 0;

        Block(Image image, int x, int y, int width, int height) {
            this.image = image;
            this.x = x;
            this.y = y;
            this.width = width;
            this.height = height;
            this.startX = x;
            this.startY = y;
        }

        void updateDirection(char direction) {
            char prevDirection = this.direction;
            this.direction = direction;
            updateVelocity();

            this.x += velocityX;
            this.y += velocityY;

            for (Block wall : walls) {
                if (collision(this, wall)) {
                    this.x -= velocityX;
                    this.y -= velocityY;
                    this.direction = prevDirection;
                    updateVelocity();
                }
            }
        }

        void updateVelocity() {
            if (direction == 'U') { velocityX = 0; velocityY = -tileSize / 4;
}
            else if (direction == 'D') { velocityX = 0; velocityY = tileSize
/ 4; }
            else if (direction == 'L') { velocityX = -tileSize / 4; velocityY
= 0; }
            else if (direction == 'R') { velocityX = tileSize / 4; velocityY
= 0; }
        }
```

```
        void reset() {
            x = startX;
            y = startY;
        }
    }

    int rowCount = 21;
    int columnCount = 19;
    int tileSize = 32;

    int boardWidth = columnCount * tileSize;
    int boardHeight = rowCount * tileSize;

    Image wallImage, blueGhostImage, orangeGhostImage, pinkGhostImage,
redGhostImage;
    Image pacmanUpImage, pacmanDownImage, pacmanLeftImage, pacmanRightImage;

    String[] tileMap = {
            "XXXXXXXXXXXXXXXXXXX",
            "X        X        X",
            "X XX XXX X XXX XX X",
            "X                 X",
            "X XX X XXXXX X XX X",
            "X    X       X    X",
            "XXXX XXXX XXXX XXXX",
            "OOOX X       X XOOO",
            "XXXX X XXrXX X XXXX",
            "O       bpo       O",
            "XXXX X XXXXX X XXXX",
            "OOOX X       X XOOO",
            "XXXX X XXXXX X XXXX",
            "X        X        X",
            "X XX XXX X XXX XX X",
            "X  X     P     X  X",
            "XX X X XXXXX X X XX",
            "X    X   X   X    X",
            "X XXXXXX X XXXXXX X",
            "X                 X",
            "XXXXXXXXXXXXXXXXXXX"
    };

    HashSet<Block> walls, foods, ghosts;
    Block pacman;

    Timer gameLoop;
    Random random = new Random();
    char[] directions = {'U', 'D', 'L', 'R'};

    int score = 0;
    int lives = 3;
    boolean gameOver = false;

    public PacMan() {
        setPreferredSize(new Dimension(boardWidth, boardHeight));
        setBackground(Color.BLACK);
        addKeyListener(this);
```

```
        setFocusable(true);

        // Load images
        wallImage = new ImageIcon("src/wall.png").getImage();
        blueGhostImage = new ImageIcon("src/blueGhost.png").getImage();
        orangeGhostImage = new ImageIcon("src/orangeGhost.png").getImage();
        pinkGhostImage = new ImageIcon("src/pinkGhost.png").getImage();
        redGhostImage = new ImageIcon("src/redGhost.png").getImage();

        pacmanUpImage = new ImageIcon("src/pacmanUp.png").getImage();
        pacmanDownImage = new ImageIcon("src/pacmanDown.png").getImage();
        pacmanLeftImage = new ImageIcon("src/pacmanLeft.png").getImage();
        pacmanRightImage = new ImageIcon("src/pacmanRight.png").getImage();

        // Load map
        loadMap();

        // Start game loop
        gameLoop = new Timer(40, this); // ~25 FPS
        gameLoop.start();
    }

    public void loadMap() {
        walls = new HashSet<>();
        foods = new HashSet<>();
        ghosts = new HashSet<>();

        for (int r = 0; r < rowCount; r++) {
            for (int c = 0; c < columnCount; c++) {
                char tile = tileMap[r].charAt(c);
                int x = c * tileSize;
                int y = r * tileSize;

                if (tile == 'X') walls.add(new Block(wallImage, x, y,
tileSize, tileSize));
                else if (tile == 'b') ghosts.add(new Block(blueGhostImage, x,
y, tileSize, tileSize));
                else if (tile == 'o') ghosts.add(new Block(orangeGhostImage,
x, y, tileSize, tileSize));
                else if (tile == 'p') ghosts.add(new Block(pinkGhostImage, x,
y, tileSize, tileSize));
                else if (tile == 'r') ghosts.add(new Block(redGhostImage, x,
y, tileSize, tileSize));
                else if (tile == 'P') pacman = new Block(pacmanRightImage, x,
y, tileSize, tileSize);
                else if (tile == ' ') foods.add(new Block(null, x + 14, y +
14, 4, 4));
            }
        }

        // Initialize ghost directions
        for (Block ghost : ghosts) {
            ghost.updateDirection(directions[random.nextInt(4)]);
        }
    }

    public void paintComponent(Graphics g) {
```

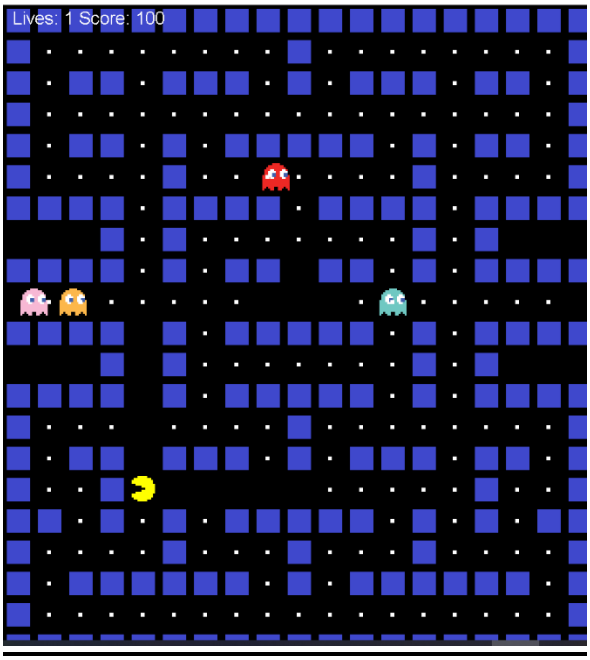
```
        super.paintComponent(g);

        g.drawImage(pacman.image, pacman.x, pacman.y, pacman.width,
pacman.height, null);

        for (Block ghost : ghosts)
            g.drawImage(ghost.image, ghost.x, ghost.y, ghost.width,
ghost.height, null);

        for (Block wall : walls)
            g.drawImage(wall.image, wall.x, wall.y, wall.width, wall.height,
null);

        g.setColor(Color.WHITE);
        for (Block food : foods)
            g.fillRect(food.x, food.y, food.width, food.height);

        g.setFont(new Font("Arial", Font.PLAIN, 18));
        g.drawString("Lives: " + lives + " Score: " + score, 10, 20);
    }

    public void movePacMan() {
        pacman.x += pacman.velocityX;
        pacman.y += pacman.velocityY;

        for (Block wall : walls) {
            if (collision(pacman, wall)) {
                pacman.x -= pacman.velocityX;
                pacman.y -= pacman.velocityY;
            }
        }

        Block eaten = null;
        for (Block food : foods) {
            if (collision(pacman, food)) {
                eaten = food;
                score += 10;
            }
        }
        if (eaten != null) foods.remove(eaten);
    }

    public boolean collision(Block a, Block b) {
        return a.x < b.x + b.width &&
                a.x + a.width > b.x &&
                a.y < b.y + b.height &&
                a.y + a.height > b.y;
    }

    public void actionPerformed(ActionEvent e) {
        if (gameOver) return;

        movePacMan();

        // Ghost movement with simple chasing AI
        for (Block ghost : ghosts) {
            // Occasionally chase PacMan
```

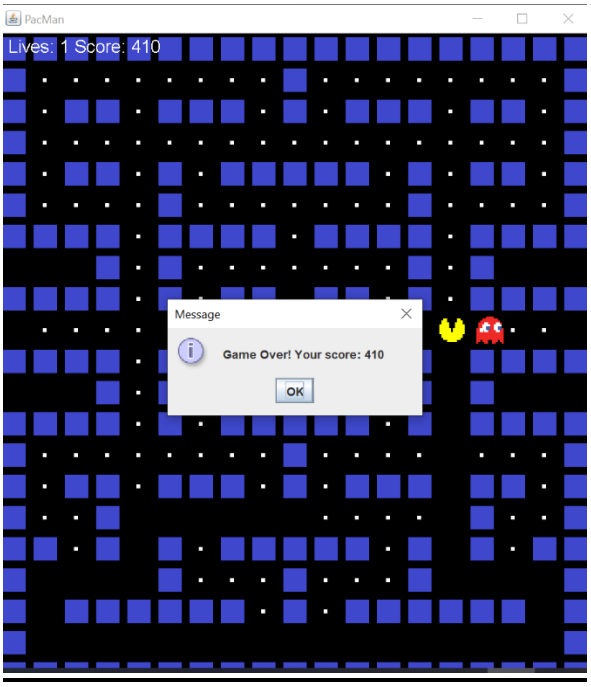
```
            if (random.nextInt(10) == 0) {
                if (Math.abs(pacman.x - ghost.x) > Math.abs(pacman.y -
ghost.y)) {
                    if (pacman.x > ghost.x) ghost.updateDirection('R');
                    else ghost.updateDirection('L');
                } else {
                    if (pacman.y > ghost.y) ghost.updateDirection('D');
                    else ghost.updateDirection('U');
                }
            }

            // Move ghost
            ghost.x += ghost.velocityX;
            ghost.y += ghost.velocityY;

            // Collision with walls
            for (Block wall : walls) {
                if (collision(ghost, wall)) {
                    ghost.x -= ghost.velocityX;
                    ghost.y -= ghost.velocityY;
                    ghost.updateDirection(directions[random.nextInt(4)]);
                }
            }

            // Check collision with PacMan
            if (collision(ghost, pacman)) {
                lives--;
                pacman.reset(); // reset PacMan
                for (Block g : ghosts) g.reset(); // reset ghosts

                if (lives <= 0) {
                    gameOver = true;
                    gameLoop.stop();
                    JOptionPane.showMessageDialog(this, "Game Over! Your
score: " + score);
                }
            }
        }

        repaint();
    }

    public void keyPressed(KeyEvent e) {
        switch (e.getKeyCode()) {
            case KeyEvent.VK_UP: pacman.updateDirection('U'); break;
            case KeyEvent.VK_DOWN: pacman.updateDirection('D'); break;
            case KeyEvent.VK_LEFT: pacman.updateDirection('L'); break;
            case KeyEvent.VK_RIGHT: pacman.updateDirection('R'); break;
        }

        switch (pacman.direction) {
            case 'U': pacman.image = pacmanUpImage; break;
            case 'D': pacman.image = pacmanDownImage; break;
            case 'L': pacman.image = pacmanLeftImage; break;
            case 'R': pacman.image = pacmanRightImage; break;
        }
    }
```

```
    public void keyReleased(KeyEvent e) {}
    public void keyTyped(KeyEvent e) {}

    public static void main(String[] args) {
        JFrame frame = new JFrame("PacMan");
        PacMan game = new PacMan();

        frame.add(game);
        frame.pack();
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);

        game.requestFocusInWindow();
    }
}
```

IMAGES:

OUTPUT:




PacMan
Lives: 1 Score: 410
Message
Game Over! Your score: 410
OK